

UNIVERSITÉ DE LILLE
FACULTÉ DES SCIENCES ET TECHNOLOGIES

Comparaison entre les approches classiques et les approches neuronales pour l'apprentissage par renforcement

Enzo Claeysen

Licence Mathématiques-Informatique



DÉPARTEMENT D'INFORMATIQUE
Faculté des Sciences et Technologies

juillet, 2025

Indice

1	Explications et introduction des algorithmes étudiés	5
1.1	L'apprentissage par renforcement	5
1.2	Le Q-Learning	8
1.3	Deep Q Network (DQN)	14
2	Codage des problèmes	19
2.1	Introduction des problèmes traités	19
2.2	Gymnasium	22
2.3	La distribution des récompenses	22
2.4	Les observations	22
3	Évaluer les performances	25
4	Comparaison effective	27
4.1	Couloir	27
4.2	Labyrinthe	32
4.3	Livraison	40
5	Jeux avec adversaires	43
6	Conclusion	45
6.1	QLearning et DQN	45
6.2	StableBaselines et RLlib	46

Chapitre 1

Explications et introduction des algorithmes étudiés

1.1 L'apprentissage par renforcement

L'apprentissage par renforcement est l'une des trois principales méthodes d'apprentissage automatique. Son principe repose sur l'introduction d'un agent autonome au sein d'un environnement.

Le plus simple est d'expliquer par un exemple.

1.1.1 Exemple d'environnement

Nous avons 4 cases, la 1ère est la case où apparaît l'agent, la dernière comporte un cookie. Le but est que l'agent récupère le cookie, mais un obstacle (un trou) se situe sur la 3ème case : 1.1

Deux actions sont mises à disposition de l'agent : Avancer ou Sauter.

Si l'agent décide d'avancer, alors il avance d'une case.

Si l'agent décide de sauter, alors soit :

- Il réussit son saut et avance directement de deux cases
- Il rate son saut et avance d'une unique case.

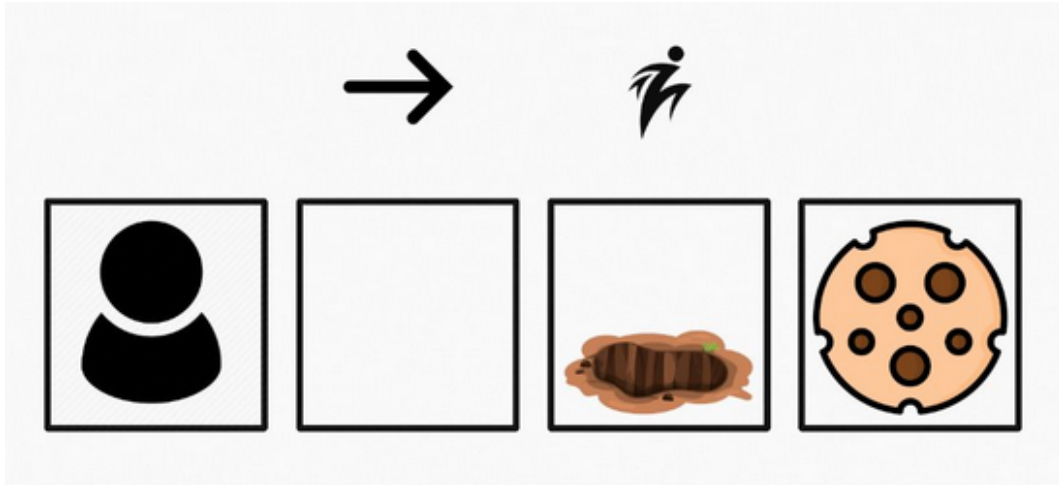


FIGURE 1.1 – Exemple Environnement

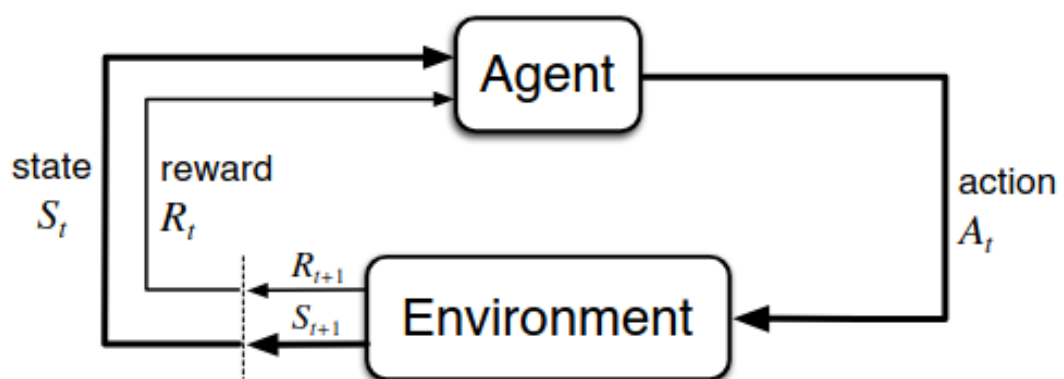


FIGURE 1.2 – Interaction Agent/Environnement

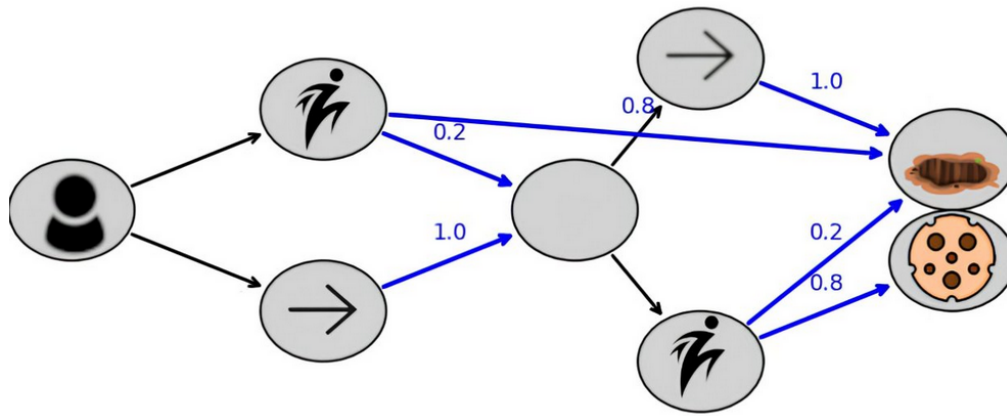


FIGURE 1.3 – Graphe de l'exemple

1.1.2 Interaction Agent-Environnement

L'agent interagit avec l'environnement en effectuant ceci : 1.2

- Il effectue une première observation
- Puis sélectionne l'action qu'il souhaite effectuer
- L'environnement modifie son état et envoie une nouvelle observation
- De plus, l'environnement envoie à l'agent une récompense selon les conséquences de l'action.

Faire ce cycle une fois, c'est effectuer un step.

La récompense est un nombre qui mesure la désirabilité de la conséquence de l'action effectuée par l'agent.

Par exemple, si une action permet à l'agent d'atteindre le cookie, alors il pourrait obtenir une récompense de 1.

Au contraire, si une action amène l'agent à tomber dans le trou, alors il pourrait obtenir une récompense négative telle que -1.

Si la conséquence n'est ni favorable, ni défavorable, alors l'agent obtiendra une récompense de 0 (neutre).

L'agent effectue alors une séquence de steps l'amenant d'un état initial à un état final à partir duquel il cesse d'effectuer des steps.

Cette séquence est appelée épisode.

1.1.3 Représentation par Graphe

Un environnement se représente par un graphe comme celui-ci : 1.3

On remarque que les états de l'environnement sont représentés par certains sommets.

Les "sommets-états" non finaux possèdent des arcs noirs sortants, ces arcs représentent les décisions qui peuvent être prises par l'agent.

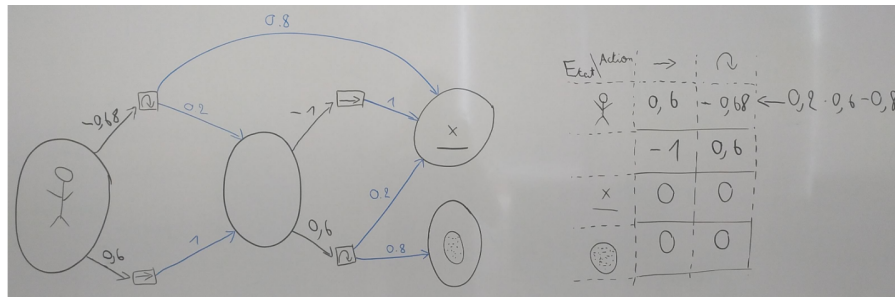


FIGURE 1.4 – QLearning utilisation de la table

Ces arcs noirs amènent à des "sommets-actions" desquels sortent des arcs bleus qui représentent les conséquences possibles des actions.

Les arcs bleus sont pondérés par la probabilité que cette conséquence se réalise.

L'objectif de l'apprentissage par renforcement est de trouver le comportement qui maximise le gain moyen.

On nomme comportement la fonction qui à chaque sommet-état associe une action possible.

Le gain est la somme des récompenses obtenues au cours d'un épisode.

On parle de gain moyen car les conséquences des actions sont parfois imprévisibles, un même comportement peut donc amener à divers gains selon l'épisode.

Une difficulté supplémentaire réside dans le fait qu'au début de son apprentissage, l'agent ne connaît pas le graphe.

Il est donc impossible de connaître les conséquences des diverses actions (arcs bleus) ainsi que les chances qu'elles surviennent (leur pondération) à moins d'interagir avec l'environnement.

1.2 Le Q-Learning

Le Q-Learning tabulaire est un algorithme d'apprentissage par renforcement créé par [Watkins, 1989]. Son principe est de maximiser le gain moyen qu'obtiendra l'agent s'il effectue l'action a sous l'état s de l'environnement.

Pour ce faire, le Q-Learning est conçu de façon à construire la fonction "Qualité" (fonction Q) qui à une paire (état, action) associe le gain moyen qu'obtiendra l'agent s'il applique cette action dans cet état.

Or l'agent n'a pas accès au graphe de l'environnement, nous allons devoir créer une estimation de cette fonction que l'on affine de steps en steps.

Cette méthode est dite "tabulaire" car l'estimation de la fonction Q est représentée par une table.

1.2.1 Comment utiliser cette table ?

Avant de comprendre comment trouver cette fonction qualité, il faut se demander comment nous pouvons en faire usage afin de résoudre notre problème.

Voici un exemple de table Q (une estimation de la fonction Q) pour le problème 1.3 : 1.4.

Nous y voyons la table qui représente l'estimation actuelle de la fonction Q. On remarque qu'il est possible de reporter les estimations des gains moyens en tant que pondération des arcs noirs du graphe de l'environnement.

À l'issue de l'entraînement, si l'agent se trouve dans une certaine situation (supposons, dans cet exemple, que l'agent se trouve dans le noeud à gauche), alors il suffit de regarder pour chaque arc noir sortant quel est celui qui possède la plus grande pondération.

Ici, l'agent choisit d'avancer d'une case puisque l'arc noir associé possède une pondération de 0.6, ce qui est supérieur à -0.68.

Lorsque l'agent décide de cette façon, on dit qu'il exploite : il sélectionne la meilleure action à effectuer au regard de ses connaissances actuelles.

Si nous trouvons la fonction Q, alors on connaît le gain moyen obtenu lorsque l'on effectue une action dans un état. Si l'agent exploite tout le temps en connaissant la fonction Q, alors il adoptera un comportement optimal, un des comportements qui maximise le gain moyen obtenu par l'agent.

1.2.2 Mise à jour de la table de façon itérative

Maintenant que l'on sait que connaître la fonction Q permet d'obtenir un comportement optimal, une nouvelle question se pose : Comment obtenir la fonction Q ?

Dans la figure 1.4, la fonction Q a été calculée à partir du graphe en partant des états finaux et en remontant jusqu'à l'état initial.

Néanmoins, l'agent n'a pas accès au graphe, il faut donc trouver une autre méthode.

Le gain obtenu lorsque l'on effectue une action dans un état n'est pas toujours le même, nous pouvons considérer que c'est une variable aléatoire.

Notre objectif : trouver son espérance (ce qui revient à trouver le gain moyen obtenu en effectuant cette action dans cet état).

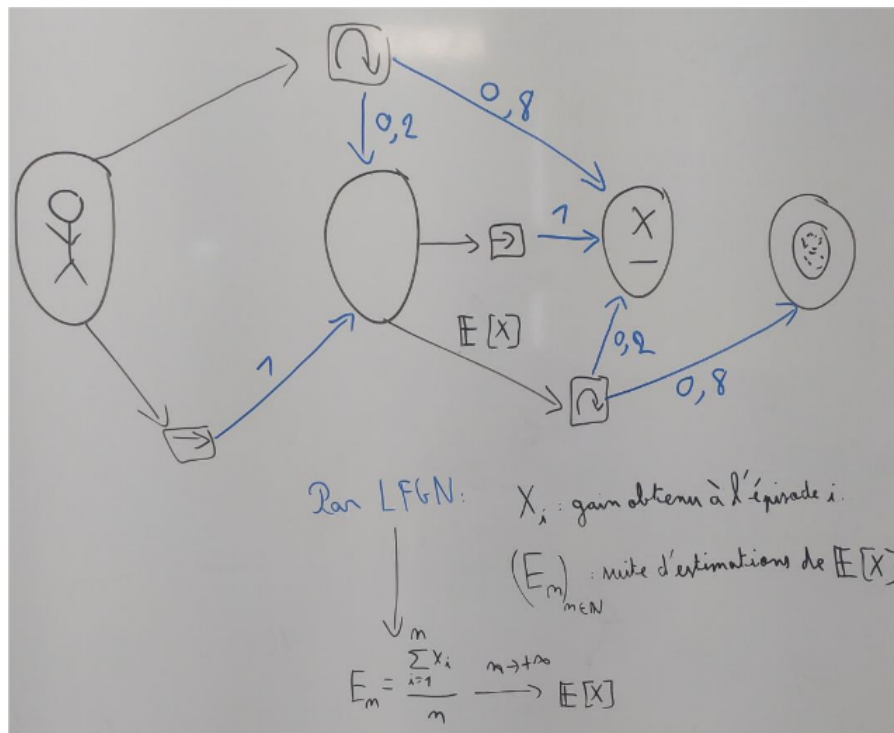


FIGURE 1.5 – Loi Forte des Grands Nombres pour trouver le poids de l'arc noir

Ainsi pour chaque paire (état, action) on cherche à calculer l'espérance d'une variable aléatoire.

Pour faire cela, on utilise la loi forte des grands nombres : on sait que la moyenne empirique permet d'obtenir une approximation de l'espérance, voir figure 1.5.

Nous pouvons manipuler la formule de la moyenne empirique afin de l'écrire de façon récursive : 1.6.

En observant suffisamment de fois le gain obtenu en effectuant une action dans un état puis en exploitant jusqu'à la fin de l'épisode, nous devrions converger petit à petit vers le résultat de la fonction Q .

Mais lorsque l'on pense à l'implémentation, deux choses posent problème.

Tout d'abord, le $1/n$ est problématique car la convergence vers l'espérance se fait asymptotiquement sur n .

n (le nombre de fois qu'on effectue une action dans un état) peut être arbitrairement grand et $1/n$ très très proche de 0, voire même tellement proche que l'ordinateur approche $1/n$ par 0 directement.

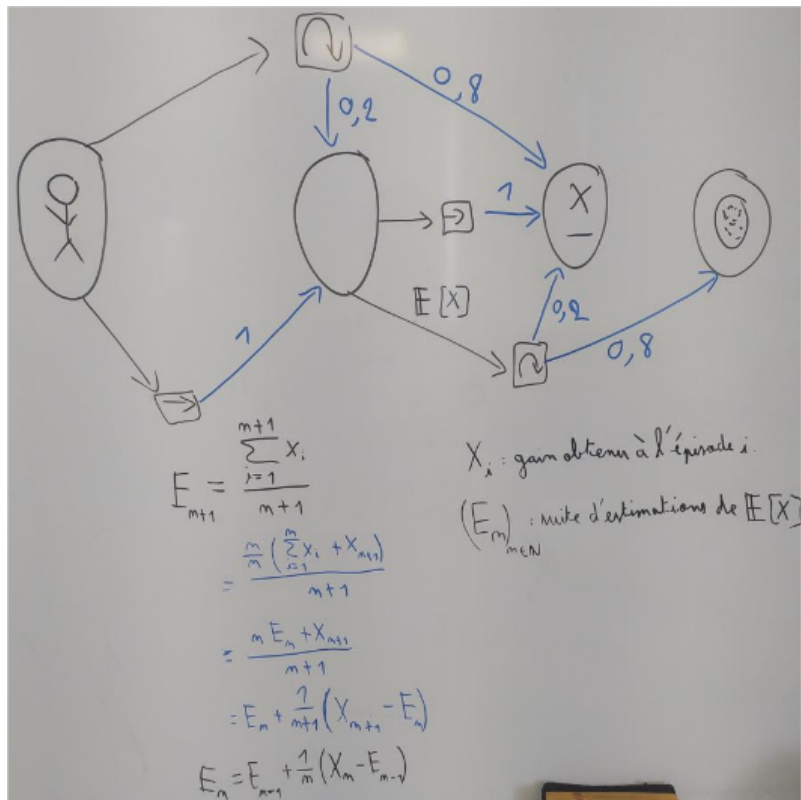


FIGURE 1.6 – Loi Forte Des Grands Nombres de façon récursive

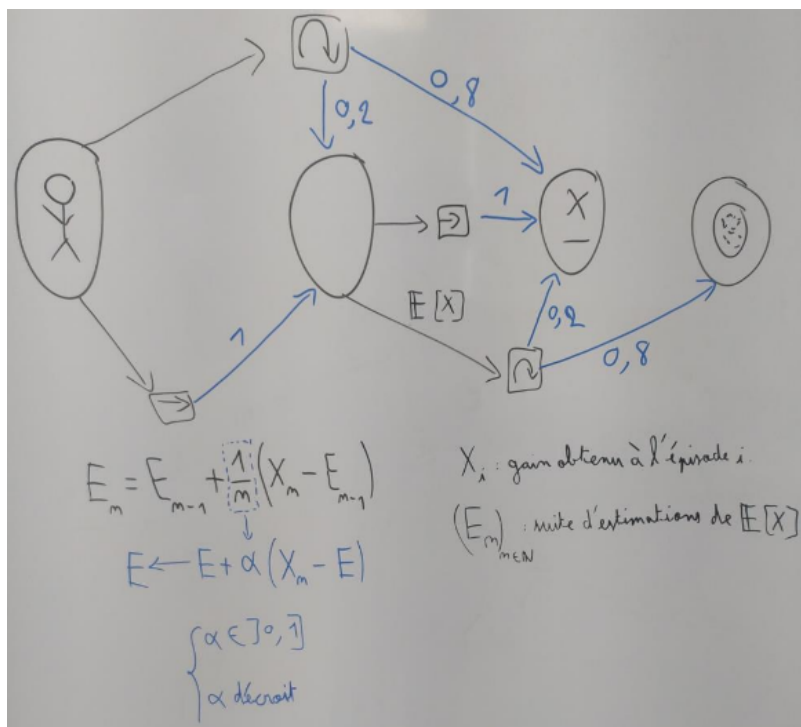


FIGURE 1.7 – Introduction du facteur d'actualisation

$$E_m = E_{m-1} + \frac{1}{m} (X_m - E_{m-1})$$

$$X_m = \sum_{i=0}^{+\infty} r_i$$

X_i : gain obtenu à l'épisode i .
 $(E_m)_{m \in \mathbb{N}}$: suite d'estimations de $\mathbb{E}[X]$
 $\alpha \in]0, 1]$
 α décroît

FIGURE 1.8 – Calcul du gain

$$E_m = E_{m-1} + \frac{1}{m} (\gamma X_m - E_{m-1})$$

$$\gamma X_m = \sum_{i=0}^{+\infty} \gamma^i r_i$$

X_i : gain obtenu à l'épisode i .
 $(E_m)_{m \in \mathbb{N}}$: suite d'estimations de $\mathbb{E}[X]$
 $\gamma \in [0, 1[$
 α décroît

FIGURE 1.9 – Introduction du facteur d'actualisation

Dans ce cas, nous aurions $E(n) = E(n-1)$, c'est-à-dire que l'agent cesse d'apprendre.

Afin de résoudre ce problème, on introduit un paramètre "learning rate" (α) qui est un coefficient qui substitue $1/n$, comme n est un naturel non nul on peut en déduire une borne pour le learning rate. (voir figure : 1.7)

Le second problème vient du fait que l'observation nécessaire au calcul de la prochaine estimation dépend du gain obtenu lors de cet épisode.

Mais un épisode peut dans certains cas être infini si l'agent se comporte d'une façon particulière, il est donc nécessaire d'apprendre au cours de l'épisode, or le gain n'est obtainable qu'à l'issue de l'épisode.

L'unique information que l'on observe est la récompense immédiate obtenue par l'agent et non le gain.

La figure 1.8 montre que le gain est la série des récompenses obtenues. Si le comportement de l'agent amène à un épisode "infini" alors cette série diverge.

Afin d'éviter que l'on ait des valeurs qui "explosent" dans la table et qui pourraient ne plus être codées dans celle-ci, on introduit un nouveau paramètre : "discount factor" (γ) qui permet de s'assurer de la convergence de cette série. (voir figure 1.9).

The image shows a handwritten version of the Bellman equation for Q-learning on a grid background. The equation is:

$$Q(s_t, a_t) \leftarrow \bar{Q}(s_t, a_t) + \alpha \left(\text{Reward} + \gamma \max_{a'} (Q(s_{t+1}, a')) - Q(s_t, a_t) \right)$$

Annotations in French:

- $\bar{Q}(s_t, a_t)$: ancienne valeur (old value)
- α : taux d'apprentissage (learning rate)
- Reward : récompense (reward)
- γ : facteur d'actualisation (discount factor)
- $\max_{a'} (Q(s_{t+1}, a'))$: meilleure action du nouvel état (best action of the new state)
- $Q(s_t, a_t)$: ~~valeur~~ ancienne valeur (old value)
- $n(s, a)$: nb de visites (number of visits) (plus j'ai visité plus c'est crédible et moins je change) (the more I visit, the more credible it is and the less I change)
- $\alpha = 0$: pas d'adaptation (no adaptation)
- $\alpha = 1$: effacement syst des anciennes valeurs (systematic erasure of old values)
- $\gamma = \frac{1}{n(s, A)}$: bien pour converger... mais bien après (good for converging... but well after)

FIGURE 1.10 – Equation de Bellman

Finalement, comme l'agent ne reçoit que la récompense immédiate et non le gain de l'épisode, nous calculons une estimation du gain que l'agent devrait obtenir à l'issue de l'épisode.

Pour cela, au lieu de faire la somme des récompenses immédiates (ce qui est impossible puisque l'on ne connaît pas les récompenses immédiates futures), on ajoute à la récompense immédiate obtenue le gain moyen estimé que l'agent devrait obtenir en effectuant la meilleure action dans l'état d'arrivée.

À chaque fois que l'agent reçoit une récompense, on met à jour la cellule (état, action) de la table qui correspond à l'aide de l'équation de Bellman : 1.10

Où

- α : Le taux d'apprentissage, qui détermine à quel point une nouvelle information est importante par rapport à ce qui est déjà appris.

- γ : facteur d'actualisation, qui détermine l'importance des récompenses futures

α et γ sont donc des paramètres à définir avant le début de l'entraînement, ils ont une très grande importance.

Attention !

Afin d'éviter de converger vers une solution sub-optimale, il faut que notre agent explore.

Comme l'agent utilise une estimation de la fonction Q , alors s'il exploite tout le temps, certaines parties du graphe de l'environnement ne seront pas explorées et l'agent peut ne pas converger vers une solution optimale.

Pour éviter cela, on fait en sorte que notre agent n'agisse pas de la même façon selon s'il s'entraîne ou non.

S'il ne s'entraîne pas alors lorsque l'agent observe l'état s , il effectue l'action a qui maximise $Q(s, a)$ (il exploite)

Cette façon de sélectionner l'action à effectuer selon ce que l'on observe et selon la fonction Q s'appelle la greedy-policy

Lorsque l'agent s'entraîne, afin de potentiellement découvrir de meilleurs comportements, il utilisera la epsilon-greedy policy.

C'est-à-dire que l'agent aura une probabilité epsilon d'effectuer une action aléatoire et une probabilité $1 - \text{epsilon}$ d'agir selon la greedy-policy. Le Q-Learning utilise 2 policy distinctes selon si l'agent s'entraîne ou non, on dit que c'est une méthode off-policy et epsilon est donc un nouveau paramètre introduit pour résoudre ce problème : si l'agent n'explore pas suffisamment lors de son entraînement, alors il peut ne jamais trouver de solution ou ne trouver qu'une solution sub-optimale.

Au contraire, si l'agent explore trop alors il perdra beaucoup de temps lors de son entraînement à effectuer des actions qui ne lui permettent pas d'apprendre quoi que ce soit d'intéressant.

1.3 Deep Q Network (DQN)

1.3.1 Présentation de l'algorithme

Et si la table Q devient trop grande ?

Cela arrive lorsque trop d'états et d'actions sont possibles.

En effet, le nombre de "cases" dans la table est de $nbEtats \cdot nbActions$ la taille de la table peut rapidement grandir.

Lorsque la table est trop grande, l'entraînement ralentit.

De plus, il se peut que la table soit trop grande pour la machine sur laquelle on utilise l'agent !

Il faut donc trouver une nouvelle façon de représenter la fonction Q .

On utilise un réseau de neurones artificiel.

DQN est donc un algorithme qui se base sur le même principe que le Q-Learning, la seule différence est le remplacement de la table par un réseau.

Cet algorithme a été créé par DeepMind en 2013 : [Volodymyr Mnih, 2013].

Voici comment on se représente un réseau "dense" (ceux que nous utilisons) : 1.11

Chaque sommet représente un neurone, chacun d'entre eux possède un biais. Chaque arête est pondérée.

Un noeud reçoit la somme des produits entre les sorties des noeuds de la couche précédente et le poids de l'arête reliant les deux sommets. À cette somme on ajoute le biais puis si on est dans une couche intermédiaire (couche cachée) on applique une fonction : la fonction d'activation. Ici on utilise la fonction

ReLU : $x \Rightarrow \max(x, 0)$

Grâce à la rétropropagation du gradient, nous pouvons changer les poids et les biais

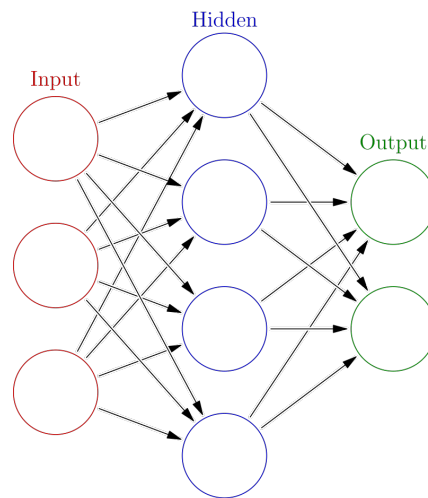


FIGURE 1.11 – Dense Neural Network

afin que le réseau renvoie des estimations les plus proches possibles des observations. Un avantage du réseau est qu'il est capable de modifier la valeur de Q pour plusieurs paires (action, état) par observation.

Néanmoins, ce phénomène est aussi sa faiblesse puisqu'il est bien plus instable.

Afin de mieux gérer cette instabilité, on utilise deux réseaux, le second réseau est appelé réseau cible. Le réseau cible sera celui utilisé lors de la mise à jour du premier réseau.

Après un certain nombre de modifications du premier réseau (le online network), on le copie et sa copie devient le nouveau réseau cible.

L'intervalle entre deux recopies est un paramètre, s'il est trop court l'apprentissage sera instable mais s'il est trop long alors l'information utilisée pour mettre à jour le réseau online sera obsolète

1.3.2 L'expérience replay

Mettre à jour le réseau est plus complexe que de mettre à jour une table.

En effet, une modification du réseau peut impacter de multiples cases de la table équivalente au réseau.

Il est donc nécessaire de prendre en compte plusieurs observations simultanément afin d'éviter de "casser" ce qui a déjà été appris en apprenant de nouvelles choses.

Afin de mieux comprendre ce processus, regardons ce schéma : 1.12

On commence par effectuer de multiples steps afin de stocker des expériences dans un buffer.

Une expérience est un tuple généré à chaque step composé de : (état de départ, action effectuée, récompense obtenue, état atteint).

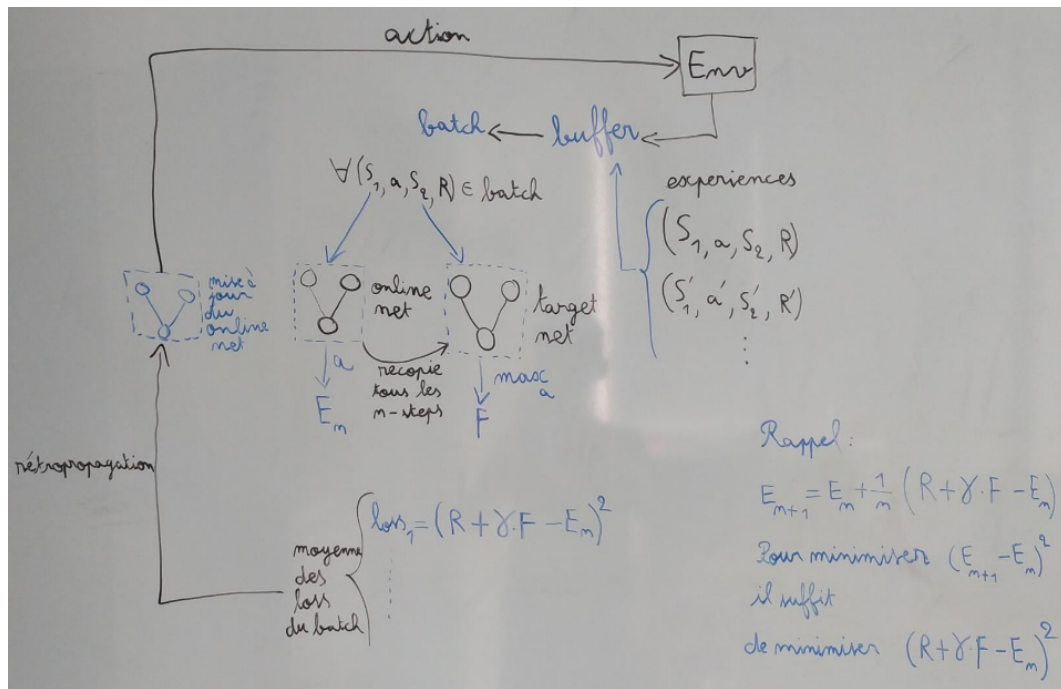


FIGURE 1.12 – DQN fonctionnement général

Le buffer est une file qui contient les x expériences les plus récentes.

Dans l'algorithme naïf, on construit un batch en choisissant de façon uniformément aléatoire des éléments du buffer.

Le batch est donc un ensemble d'expériences et pour chacune d'entre elles, on récupère l'estimation du gain moyen faite par le online network pour l'action effectuée par l'agent dans l'état initial.

C'est cette estimation (appelée E_n dans le schéma) qu'il faut changer.

On doit donc mettre à jour le online network de façon à ce qu'il renvoie $E(n+1)$ au lieu de $E(n)$ pour cette action dans cet état.

On obtient l'estimation du meilleur gain moyen obtainable dans l'état d'arrivée à l'aide du target network.

À droite vous trouverez un rappel de la formule utilisée pour calculer $E(n+1)$ à partir de $E(n)$.

Mais ici, inutile de calculer $E(n+1)$, grâce à la méthode du gradient, il suffit de réduire l'écart entre ce que renvoie le online network et $E(n+1)$.

Nous calculons une erreur (loss) pour cette expérience, minimiser cette erreur (qui est au carré) minimise la distance qui sépare ce que renvoie le online network de la cible qu'il doit atteindre.

Le problème, c'est que lors de la rétropropagation du gradient, nous ne pouvons pas considérer les expériences du batch un par un car sinon nous modifierons les sorties du réseau pour d'autres actions dans d'autres états ce qui "casse" ce qui a déjà été appris.

À la place, on calcule l'erreur au carré pour chaque expérience du batch puis la rétropropagation se fait sur la moyenne de ces erreurs.

Cela permet au réseau de minimiser l'erreur par rapport à la cible à atteindre dans de multiples cas simultanément.

On peut effectuer plusieurs fois la création d'un batch puis la mise à jour des poids de online network avant d'effectuer une nouvelle action qui permet d'avoir une nouvelle expérience que l'on renseigne dans le buffer.

À intervalle régulier, le online network est copié afin de créer un nouveau target network.

De nouveaux paramètres sont donc introduits par ce fonctionnement :
la taille du batch : si elle est trop élevée alors le temps de calcul est trop long, si le batch est trop court alors l'entraînement est instable.

La taille du buffer dépend surtout de la taille du batch, il doit être suffisamment grand pour contenir de multiples épisodes, cela permet d'éviter que le contexte des épisodes stockés biaise les expériences sur lesquelles l'entraînement se fait.

L'intervalle de recopie du online network pour mettre à jour le target network ne doit pas être trop court car dans ce cas l'entraînement est instable, mais si l'intervalle est trop long alors l'approximation de $E(n+1)$ sera trop grossière.

Il faut aussi essayer de multiples architectures pour les deux réseaux, changer le nombre de couches intermédiaires ainsi que le nombre de noeuds que chacune de ces couches contient.

De plus, il faut faire attention à ne pas confondre le learning rate du QLearning qui permet de calculer $E(n+1)$ et le learning rate de DQN qui correspond à la marche du gradient.

Pour rappel, il n'est pas nécessaire de calculer $E(n+1)$ dans DQN puisqu'il suffit de minimiser une petite formule qui se trouve en bas à droite de la figure 1.12

Chapitre 2

Codage des problèmes

2.1 Introduction des problèmes traités

2.1.1 Couloir

Le premier problème consiste en une succession de cases. Ces cases forment un couloir. L'agent commence tout à gauche et doit arriver sur la case la plus à droite. L'agent n'observe que sa position.

Deux actions sont possibles :

- Aller à gauche
- Aller à droite

2.1.2 Labyrinthe

L'objectif est d'apprendre à un agent à résoudre un labyrinthe spécifique sauf que les murs sont remplacés par des trous.

Si l'agent tombe dans un trou, l'épisode prend fin immédiatement.

Environnement basé sur “FrozenLake” de Gymnasium Voici un exemple de labyrinthe dans son état initial (l'agent se trouve sur la case de départ) : 2.1

. : Position de l'agent

S : Point de départ

F : Case libre

H : Trou

G : L'objectif

```

.FFF
FFHF
FFHF
FHFG

```

FIGURE 2.1 – Labyrinthe de taille 4 (16 cases)

```

SFFF
FHFF
BHFF
HFFG

```

FIGURE 2.2 – Livraison de taille 4 (16 cases)

Lorsque l'agent tombe dans un trou, l'épisode prend fin.

Il faut que l'agent aille du départ jusqu'à l'objectif sans tomber.

Nous pouvons générer une carte aléatoire qui possède au moins un chemin de S à G.

L'entraînement ne se fait donc que sur une unique carte puisque l'objectif est de résoudre une carte précisément.

Il y a 4 actions possibles : se déplacer dans chacune des 4 directions.

L'agent observe au moins sa position, dans certains essais nous avons ajouté les 4 cases adjacentes à l'observation.

2.1.3 Livraison

On reprend le problème du labyrinthe, sauf que cette fois-ci on place une boîte B.

L'agent doit d'abord saisir la boîte, puis se diriger vers l'objectif.

Exemple 2.2

On remarque que l'agent doit parfois rebrousser chemin une fois avoir atteint la boîte, c'est pourquoi il est nécessaire pour l'agent de savoir s'il a déjà pris la boîte ou non.

En plus de sa position, l'agent observe s'il possède la boîte ou non.

Dans certains essais, les 4 cases adjacentes sont ajoutées aux observations.

Pour le labyrinthe et la livraison, 4 actions sont possibles pour avancer dans chacune

R - - | - - | - - X - - | - - | - - A

FIGURE 2.3 – RomeAlésia Position Initiale

R - - | - - X - - M - - | - - | - - A

FIGURE 2.4 – RomeAlésia

des 4 directions.

2.1.4 Rome Alésia

Ce jeu oppose deux joueurs.

Un joueur joue Rome, l'autre joue Alésia.

L'objectif est de capturer la ville adverse.

Pour cela, il faut faire avancer le front sur la position de la ville adverse.

En début de partie, un certain nombre d'unités est attribué aux deux joueurs.

De plus, un certain nombre d'étapes est choisi pour séparer les villes du point de départ.

Le plateau se présente ainsi (pour 2 étapes) 2.3

R : Rome

A : Alésia

| : Une étape

X : Front

Les deux joueurs misent un certain nombre d'unités simultanément.

Celui ayant le plus grand nombre fait avancer le front d'une étape vers la ville adverse.

Les deux joueurs perdent le montant misé d'unité.

Exemple : Si les deux joueurs commencent avec 50 unités.

Actions :

Rome : 10 Unités

Alésia : 20 Unités

Alésia utilise plus d'unité, le front avance vers Rome :

Visuel : figure 2.4

Unités restantes :

40 pour Rome

30 pour Alésia

On recommence jusqu'à la fin de la partie.

Il y a match nul si les deux joueurs ne possèdent plus d'unités et que le front n'atteint aucune ville.

2.2 Gymnasium

Afin d'utiliser des bibliothèques telles que `stable baselines`, il est nécessaire de créer l'environnement en se basant sur `Gymnasium`.

`Gymnasium` est une bibliothèque qui propose certains environnements, mais il est possible de créer ses propres environnements qui doivent suivre un “schéma” précis :

- Les environnements sont des classes qui héritent de `gymnasium.Env`
- Ils possèdent un constructeur `__init__()` dans laquelle il faut définir les attributs `observation_space` et `action_space`.
- Une méthode `step()` qui prend une action, l'applique à l'environnement et renvoie le nouvel état et la récompense
- Une méthode `reset()` qui prend une seed (pour les environnements stochastiques), réinitialise l'environnement et renvoie l'état initial.

2.3 La distribution des récompenses

La distribution des récompenses correspond à la façon dont l'environnement distribue les récompenses.

Nous avons constaté qu'une mauvaise distribution peut fortement impacter les performances des divers algorithmes d'apprentissage.

Par exemple, si dans le problème du labyrinthe, on ne donne pas de récompense négative à l'agent dans le cas où il tombe, alors les performances de l'apprentissage sont fortement dégradées.

Si on souhaite que l'agent trouve le chemin optimal, il faut lui attribuer une récompense négative plus modérée à chaque pas sinon il trouvera un chemin mais pas nécessairement le plus court.

Il faut aussi faire attention à ce que les récompenses négatives attribuées à chaque pas ne soient pas trop élevées sinon l'agent apprendra à courir vers le trou le plus proche plutôt que de trouver la sortie.

Une distribution des récompenses choisie correctement est donc essentielle puisque les récompenses dictent les objectifs de l'agent.

2.4 Les observations

Une autre partie à ne pas négliger est ce que l'agent peut observer.

Une bonne façon de savoir si les observations suffisent pour que l'agent puisse résoudre le problème est de se mettre à sa place.

Par exemple dans le problème du labyrinthe, si nous donnons à l'agent uniquement sa position comme information alors il ne peut qu'apprendre par essai-erreur en retenant “par coeur” le chemin à suivre.

Il n'y a donc aucun moyen pour l'agent de généraliser sa capacité à résoudre ce labyrinthe à d'autres labyrinthes puisqu'il ne possède pas suffisamment d'informations. Au-delà de la sémantique des informations envoyées, on peut aussi changer la façon dont les observations sont codées.

Pour le QLearning il n'y a pas vraiment de possibilité pour le problème du labyrinthe à part permettre à l'agent d'observer plus de cases aux alentours.

Néanmoins, pour DQN, des changements peuvent être observés.

Pour le problème de la livraison, il est possible soit de coder la position et la possession de la boîte en un unique entier, auquel cas pour un labyrinthe comportant 16 cases, il y aura 32 noeuds pour la première couche (en utilisant un one hot encoding).

Soit on code ces deux informations séparément, nous aurons 16 noeuds d'entrée pour la position et 2 noeuds pour la possession de la boîte.

On peut aussi modifier l'entrée du réseau :

On peut utiliser un "One Hot Encoding" ce qui signifie que pour une information (par exemple la position), on inclut dans la première couche du réseau autant de noeuds que de positions possibles, chaque noeud est "lié" à une position.

Puis chaque noeuds aura 0 pour sortie sauf pour l'un d'entre eux, celui qui correspond à la position actuelle de l'agent, qui aura 1 pour sortie.

Le One Hot Encoding n'est pas l'unique méthode possible, puisque dans certaines situations cela augmente trop le nombre de noeuds.

On peut aussi ne prendre qu'un unique noeud d'entrée qui aura pour sortie l'entier correspondant à la position actuelle de l'agent.

Pour chaque information à fournir à l'agent, nous pouvons choisir la méthode de notre choix et ajouter à la première couche du réseau les noeuds nécessaires.

Expérimenter avec différentes manières de coder l'information (les observations / le retour du réseau) peut permettre d'obtenir un apprentissage plus efficace.

Chapitre 3

Évaluer les performances

Notre objectif est de comparer le QLearning et deux implémentations distinctes de DQN.

La première implémentation de DQN est disponible dans la librairie StableBaselines et la seconde dans RLlib.

Afin de les comparer sur les 3 premiers problèmes (couloir, labyrinthe, livraison), il faut commencer par les paramétrer.

Le paramétrage de chaque algorithme pour chaque problème est effectué par un GridSearch : pour chaque paramètre on définit un ensemble de valeurs à essayer puis on teste les combinaisons possibles afin de savoir laquelle donne un entraînement plus performant.

Pour comparer les différentes combinaisons, on affiche le gain par épisode obtenu par l'agent.

Pendant son entraînement, l'agent effectue de multiples épisodes, à chaque fois qu'il en termine un, il obtient un gain que l'on conserve pour effectuer l'affichage.

Par moment, on affiche le gain obtenu hors entraînement : à chaque fois qu'un épisode pendant l'entraînement est terminé, on relance un épisode sur lequel l'agent n'apprend rien mais où il utilise la greedy-policy.

Cette méthode peut permettre de réduire le bruit sur les gains surtout lorsque l'agent explore beaucoup.

En effet, comme ces algorithmes sont off-policy, le comportement pendant et hors entraînement n'est pas le même et celui pendant entraînement effectue de temps à autre une action aléatoirement ce qui change le gain obtenu lors de l'épisode.

Chapitre 4

Comparaison effective

4.1 Couloir

Initialement QLearning est paramétré ainsi :

- $\alpha = 0.1$
- $\gamma = 0.9$
- valeur de départ d'epsilon = 0.9

Après le grid search :

- $\alpha = 0.9$
- $\gamma = 0.8$
- valeur de départ d'epsilon = 0.7

On remarque grâce à la figure 4.1 que le QLearning paramétré (en bleu) est bien plus efficace que celui par défaut (en orange).

Cette différence s'explique par le fait que le problème du couloir donne toujours les mêmes récompenses lorsque l'on effectue une action dans un état particulier. Cela permet d'augmenter le learning rate et donc d'accélérer l'entraînement.

Pour le paramétrage de DQN, nous avons changé :

- l'architecture des réseaux en passant de 2 couches intermédiaires de 64 noeuds à une unique couche intermédiaire de 2 noeuds - L'intervalle de recopie qui est passé

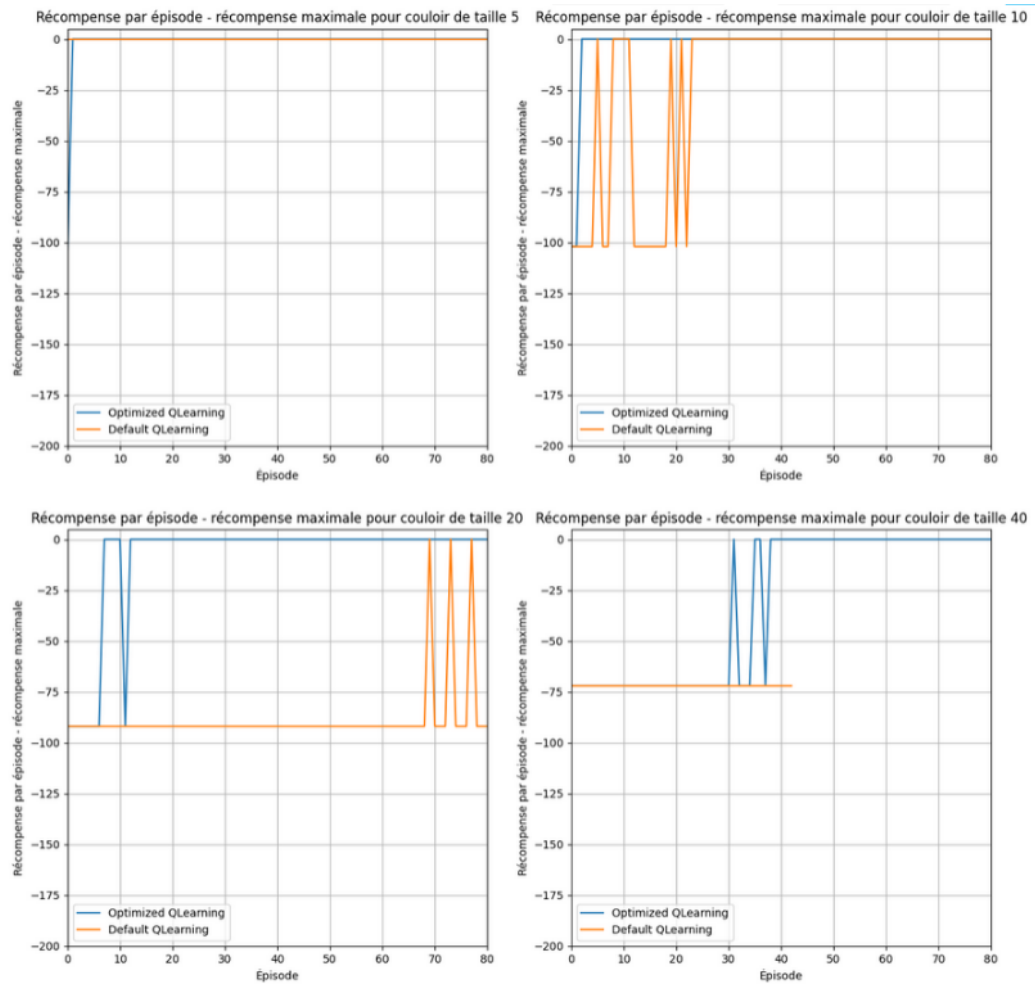


FIGURE 4.1 – Paramétrage QLearning pour Couloir

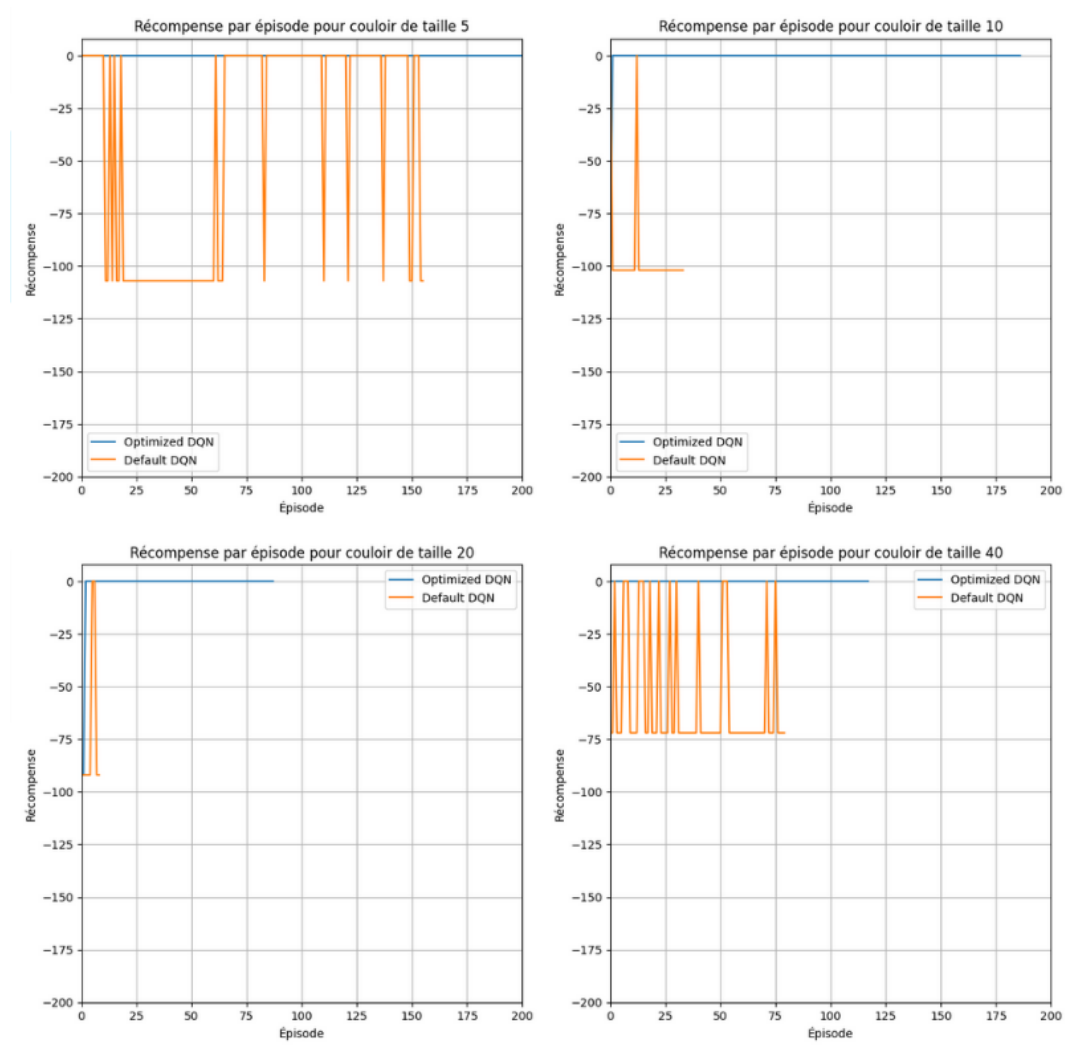


FIGURE 4.2 – Paramétrage DQN pour Couloir

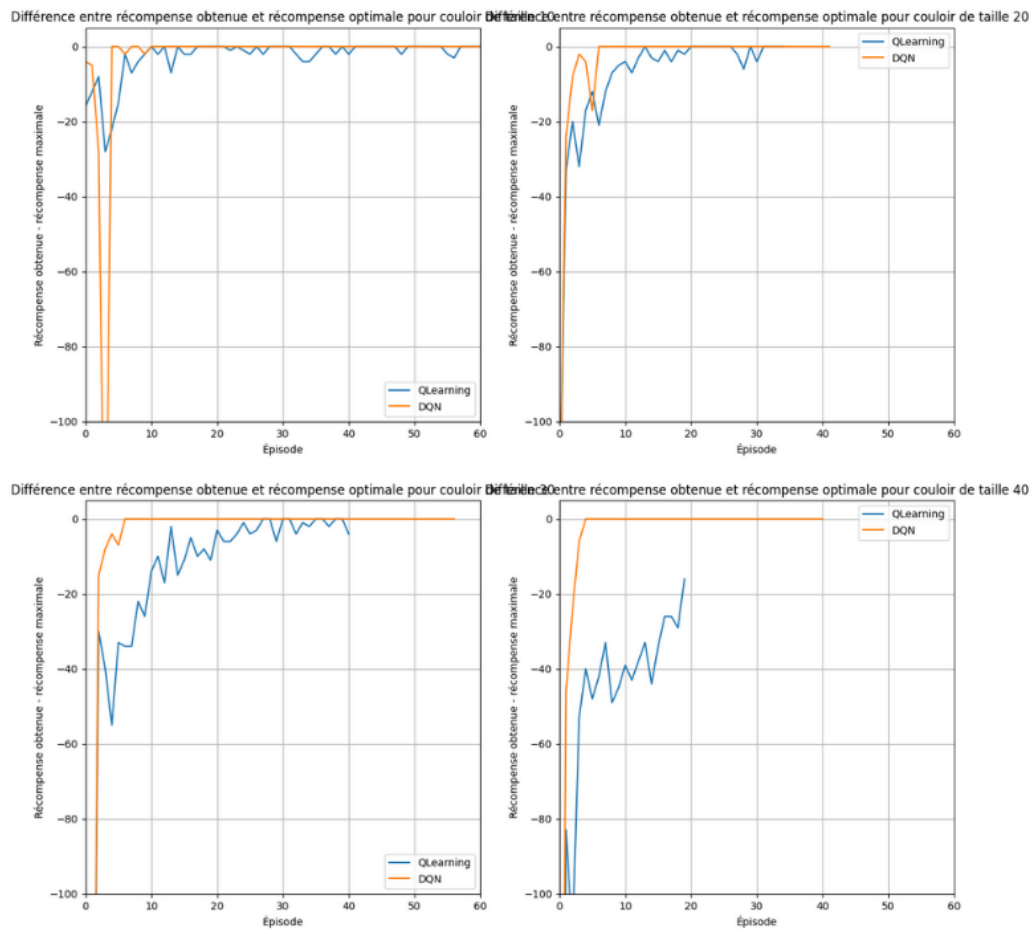


FIGURE 4.3 – Couloir comparaison QLearning/DQN

de 500 steps à 5 steps.

Le problème est très simple, il n'est pas nécessaire d'avoir un réseau complexe.

On remarque que le DQN avec les nouveaux paramètres (en bleu) est bien plus performant : 4.2

On peut comparer DQN et QLearning après les avoir paramétrés : 4.3

DQN est bien plus efficace en raison de sa capacité à généraliser.

En effet, QLearning doit apprendre à chaque case le fait d'aller à droite là où DQN peut l'apprendre pour tout le couloir d'un coup.

Jusqu'à présent le DQN comparé au QLearning était celui de StableBaselines, mais il y a aussi de RLlib.

On remarque ici : 4.4, que l'implémentation de DQN de RLlib n'est vraiment pas efficace.

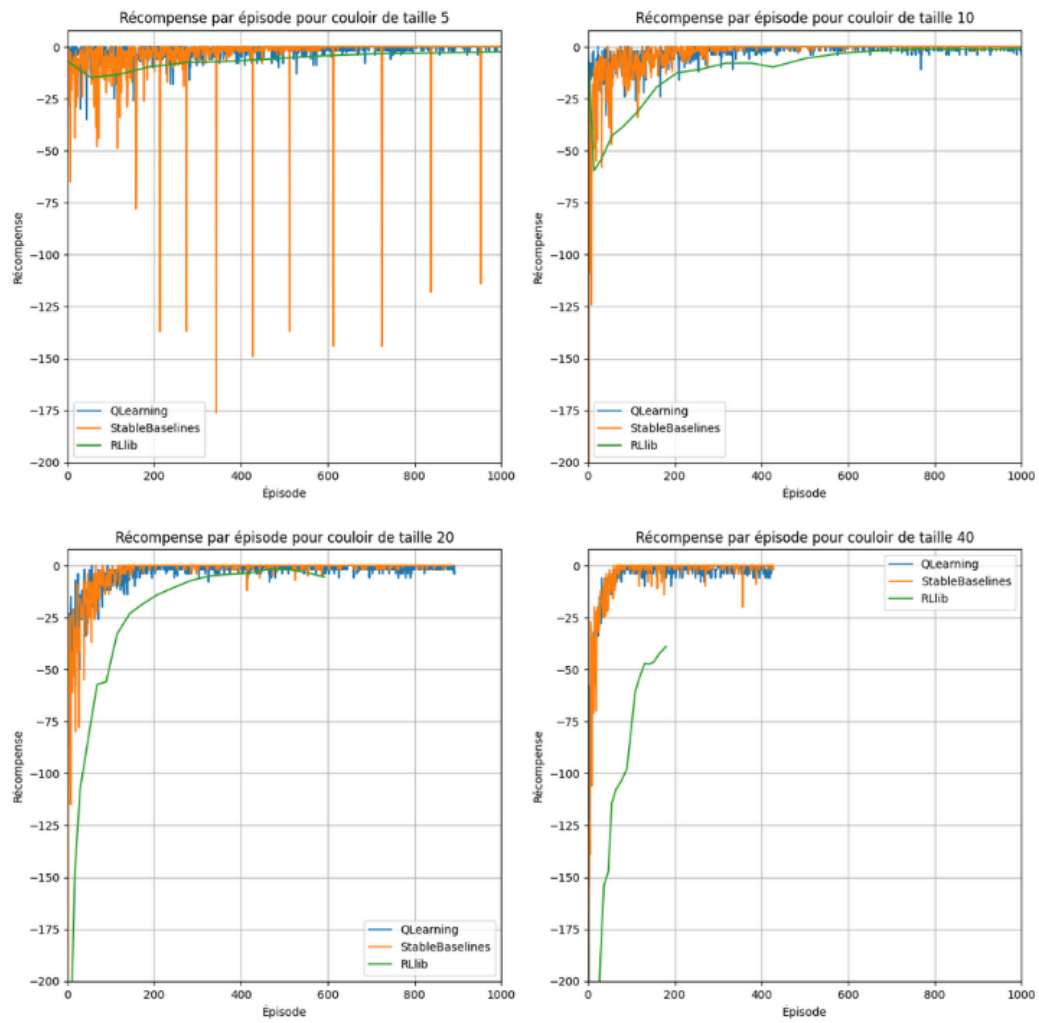


FIGURE 4.4 – Couloir dernière comparaison

4.2 Labyrinthe

Plusieurs versions du labyrinthe ont été créées.

Pour chaque version du labyrinthe, et pour chaque taille de labyrinthe étudiée, un gridSearch a été effectué pour les deux DQN.

La "taille" du labyrinthe correspond à la longueur d'un côté, donc un labyrinthe taille 4 est un carré de 16 cases.

4.2.1 Labyrinthe 1

Dans cette première version du labyrinthe, l'attribution des récompenses se fait ainsi :

- 0 tout le temps sauf
- 1 lorsque la sortie est trouvée

On remarque que même les performances du QLearning sont mitigées voir 4.5. Cela est dû à un problème dans l'attribution des récompenses.

Il faudrait que l'agent ait une pénalité s'il tombe dans un trou et une pénalité pour chaque déplacement.

La pénalité pour le fait de tomber permet d'apprendre plus rapidement le fait de ne pas tomber dans un trou.

La pénalité à chaque déplacement permet de s'assurer que l'agent favorisera les chemins les plus courts.

Néanmoins, si la pénalité à chaque déplacement est trop forte (récompense trop négative), alors l'agent se déplacera vers le trou le plus proche.

4.2.2 Labyrinthe 2

Dans cette seconde version du labyrinthe, l'attribution des récompenses se fait ainsi :

- 1 pour trouver la sortie
- -1 si l'agent tombe dans un trou
- -1/nombreCases dans les autres cas

Pour obtenir ces bons résultats : 4.6, nous avons augmenté le learning rate de 0.1 à 0.9 et défini epsilon à 0.05 constant au lieu de le faire décroître au fil de l'entraînement.

Si l'agent tombe dans un trou, l'épisode prend fin et il obtient immédiatement une récompense négative ; un haut learning rate permet d'apprendre plus rapidement à esquiver les trous dans lesquels l'agent tombe.

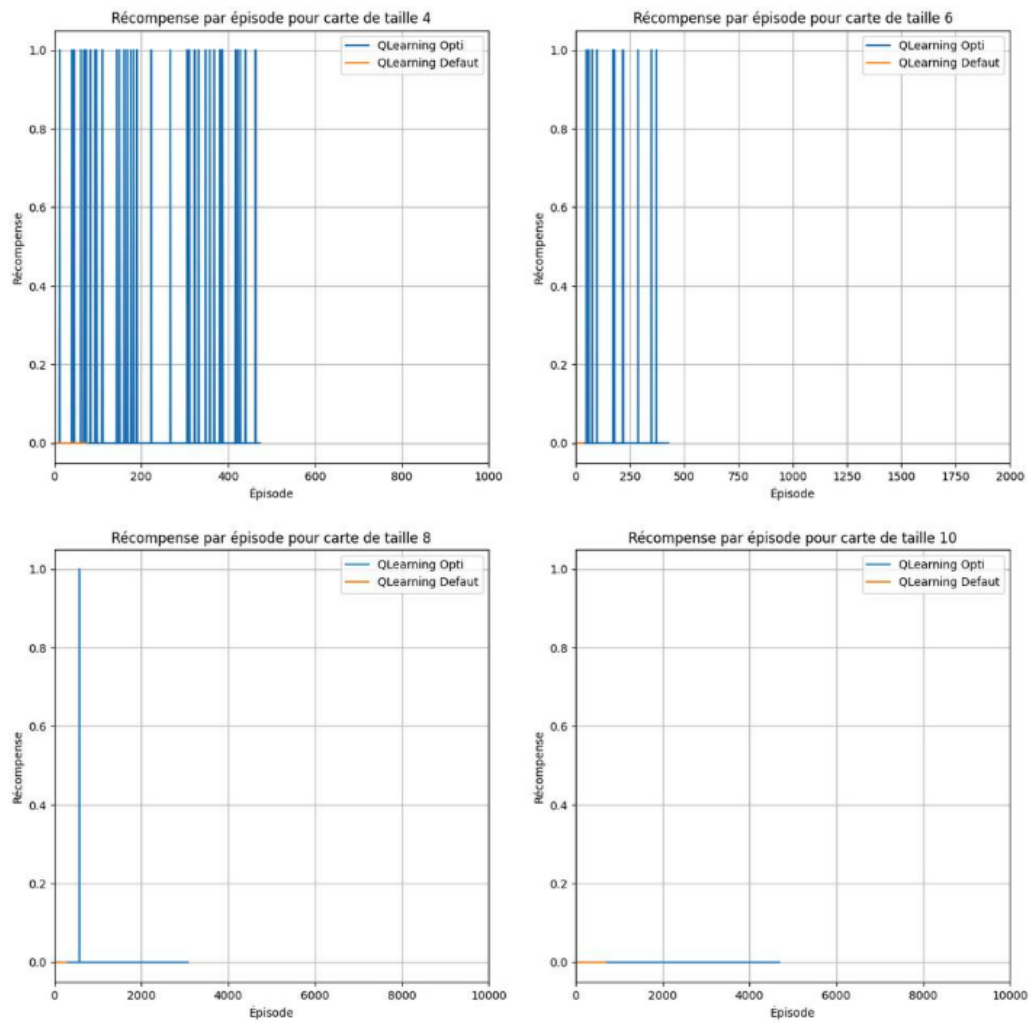


FIGURE 4.5 – QLearning pour labyrinthe 1

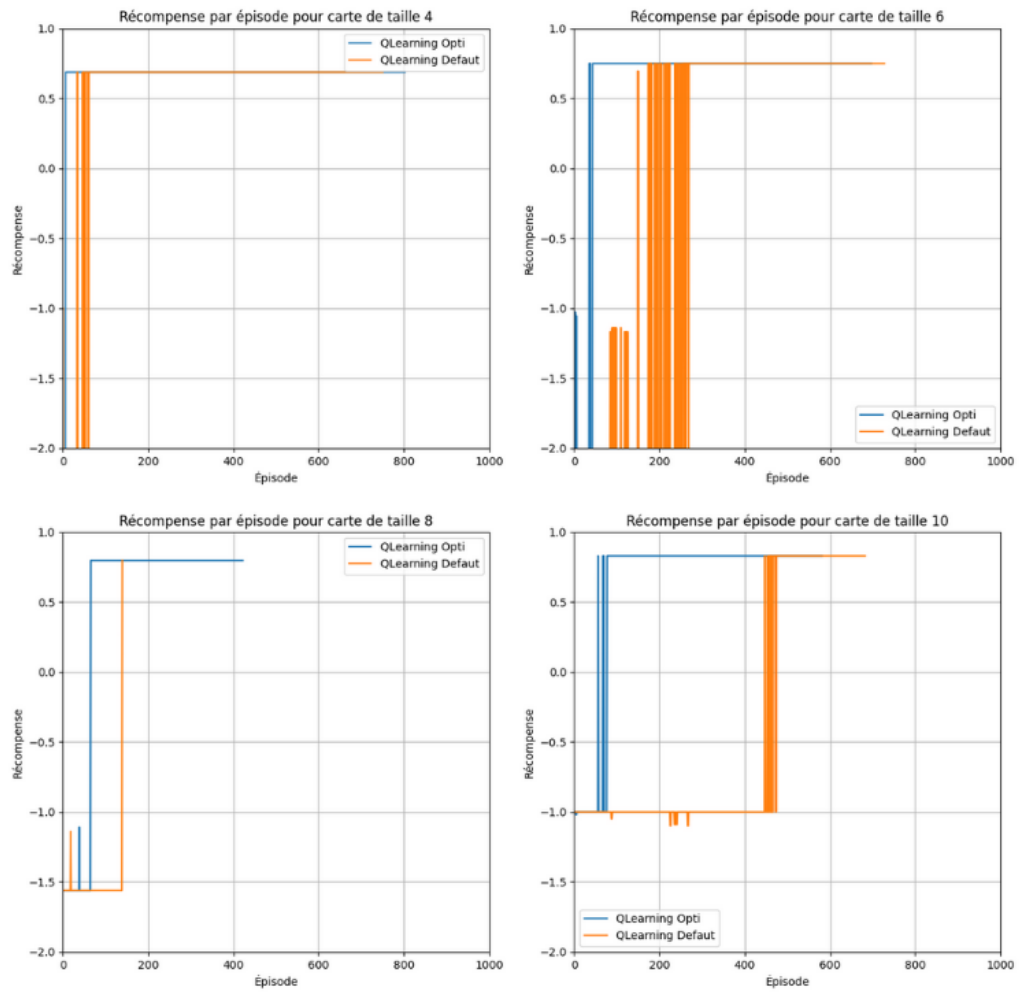


FIGURE 4.6 – QLearning labyrinthe 2

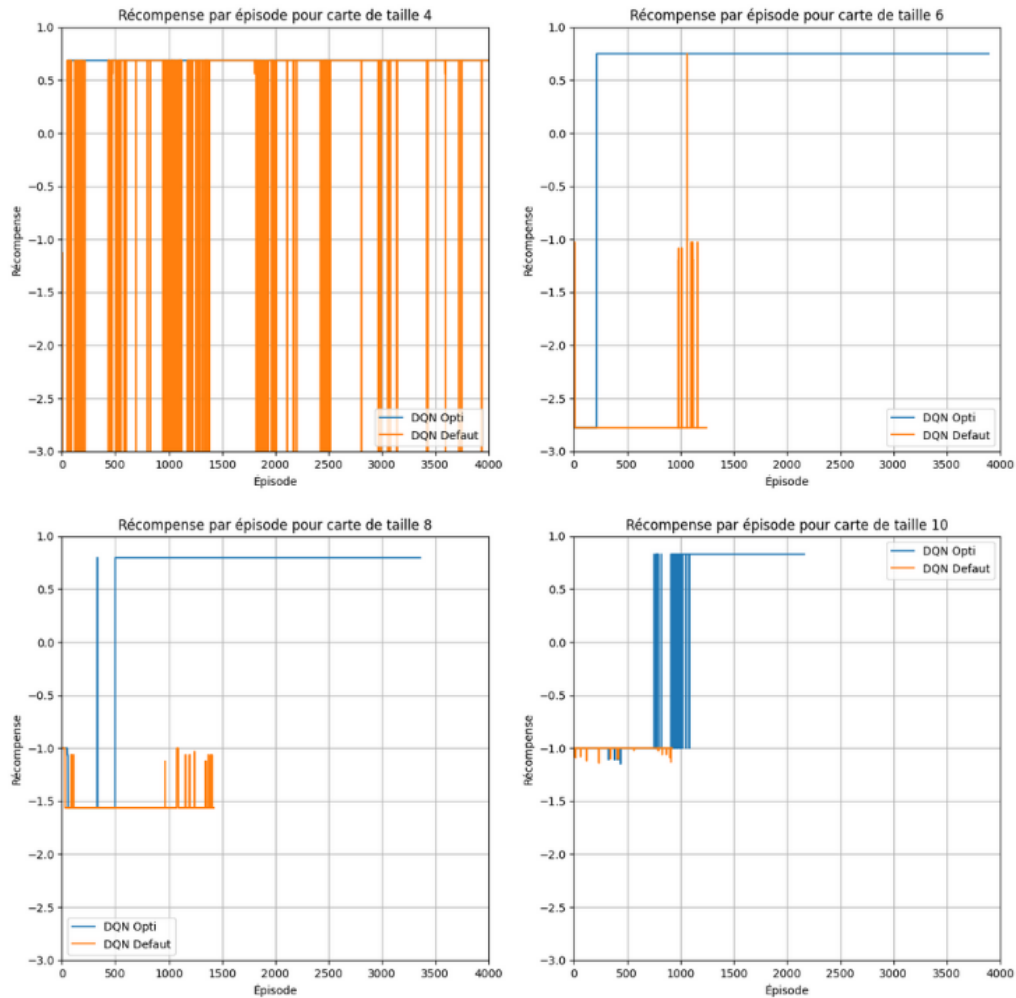


FIGURE 4.7 – DQN labyrinthe 2

L'amélioration des récompenses obtenues lors d'une faible exploration peut s'expliquer par le fait que l'agent est pénalisé à chaque déplacement alors que l'initialisation des cases de la table se fait à 0.

L'agent va donc choisir à chaque épisode d'entraînement un nouveau chemin puisque l'initialisation est plus élevée que la récompense immédiate obtenue à chaque déplacement.

Le plus gros changement dans les paramètres du DQN est une forte augmentation de la taille du batch et du buffer.

DQN a besoin d'avoir dans son batch des expériences desquelles il peut généraliser quelque chose.

Or, dans le problème du labyrinthe, l'agent n'observe que sa position, il est donc complexe d'en déduire quoi que ce soit sur l'action à effectuer.

Pour éviter de casser ce que l'agent a déjà appris, il faut considérer énormément

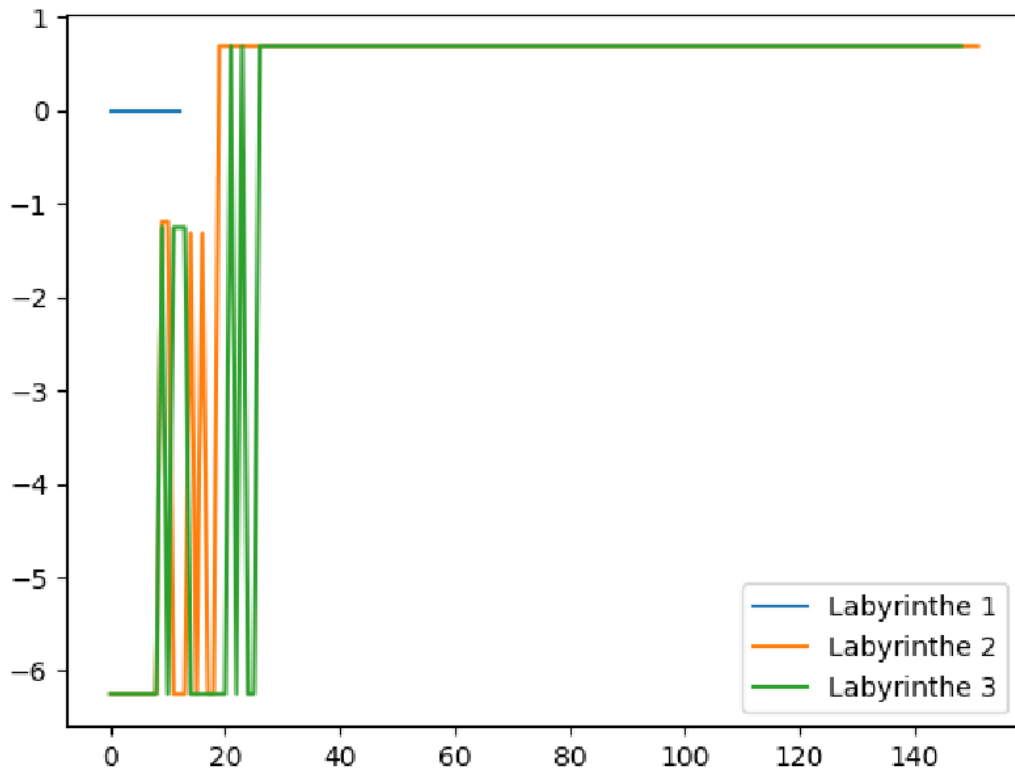


FIGURE 4.8 – QLearning labyrinth 3

d'expériences dans le batch, la taille du batch est donc élevée ce qui nécessite une augmentation de la taille du buffer.

4.2.3 Labyrinthe 3

Dans cette troisième version de labyrinthe, en plus d'observer sa position, l'agent observe les 4 cases adjacentes.

Chaque information (Position, état voisin 1, état voisin 2, état voisin 3, état voisin 4) se code par un OneHotEncoding.

Pour le labyrinthe 3, le meilleur paramétrage du QLearning est le même que pour le labyrinthe 2.

De plus, on observe 4.8 que le QLearning a les mêmes performances.

Cela est explicable par le fait que les cases adjacentes à une case fixée du labyrinthe sont toujours les mêmes.

Comme le QLearning considère chaque position de façon indépendante, l'état des voisins n'est pas une information prise en compte.

Plus précisément, la taille de la QTable sera plus grande pour le labyrinthe 3, mais

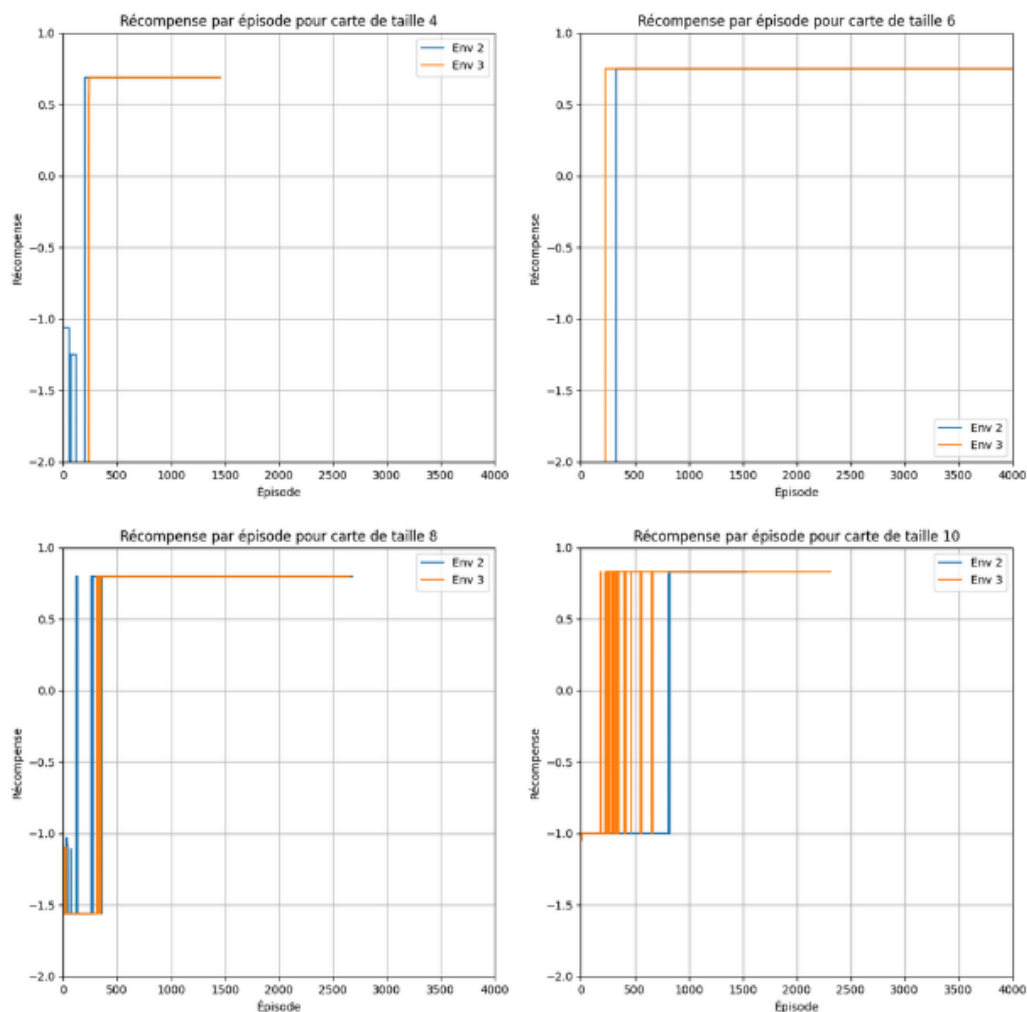


FIGURE 4.9 – DQN labyrinthe 3

les cases exploitées dans les deux tables seront les mêmes.

Ce qui est étonnant, c'est que nous observons la même chose pour DQN : 4.9 Les paramètres optimaux sont les mêmes et les performances aussi alors que grâce à cette nouvelle information on pourrait s'attendre à ce que DQN esquive plus facilement les trous.

Peut-être qu'il y a une différence mais qu'elle n'est pas flagrante car dans les labyrinthes créés, il n'y a qu'une probabilité de 0.2 qu'une case soit un trou. Il y a donc peut-être pas suffisamment de trous pour que cette information supplémentaire soit réellement utile.

Afin de s'en assurer, j'ai essayé de paramétrer de nouveaux sur des cartes où les cases ont une probabilité de 0.5 d'être un trou mais cela rend l'atteinte de l'objectif

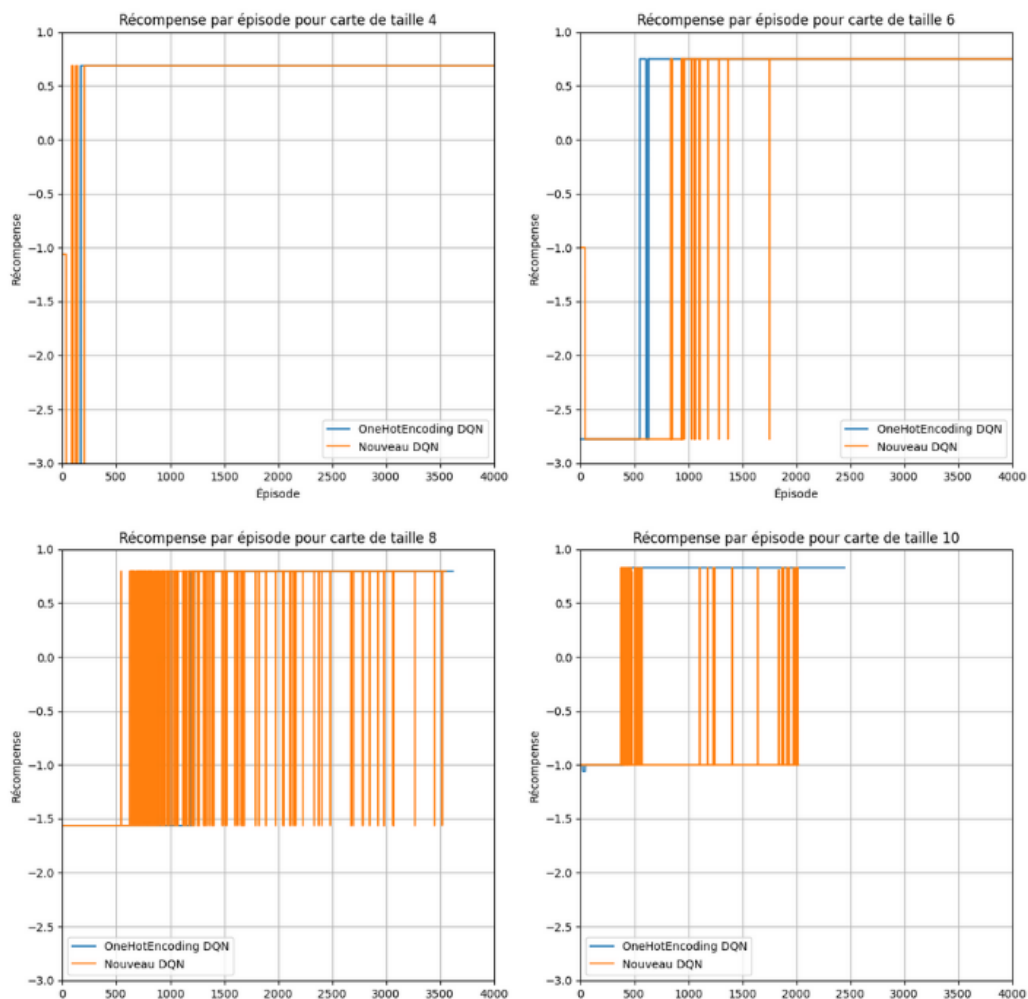


FIGURE 4.10 – DQN labyrinthe 4

tellement complexe que DQN n'arrive plus à trouver le chemin.

Ce problème n'est tout simplement pas adapté à DQN.

4.2.4 Labyrinthe 4

Dans cette dernière version du labyrinthe, l'agent observe sa position et les cases adjacentes.

Mais la position de l'agent est codée par deux noeuds chacun associé à une coordonnée.

Nous remarquons que les résultats ne sont pas très bons : 4.10.

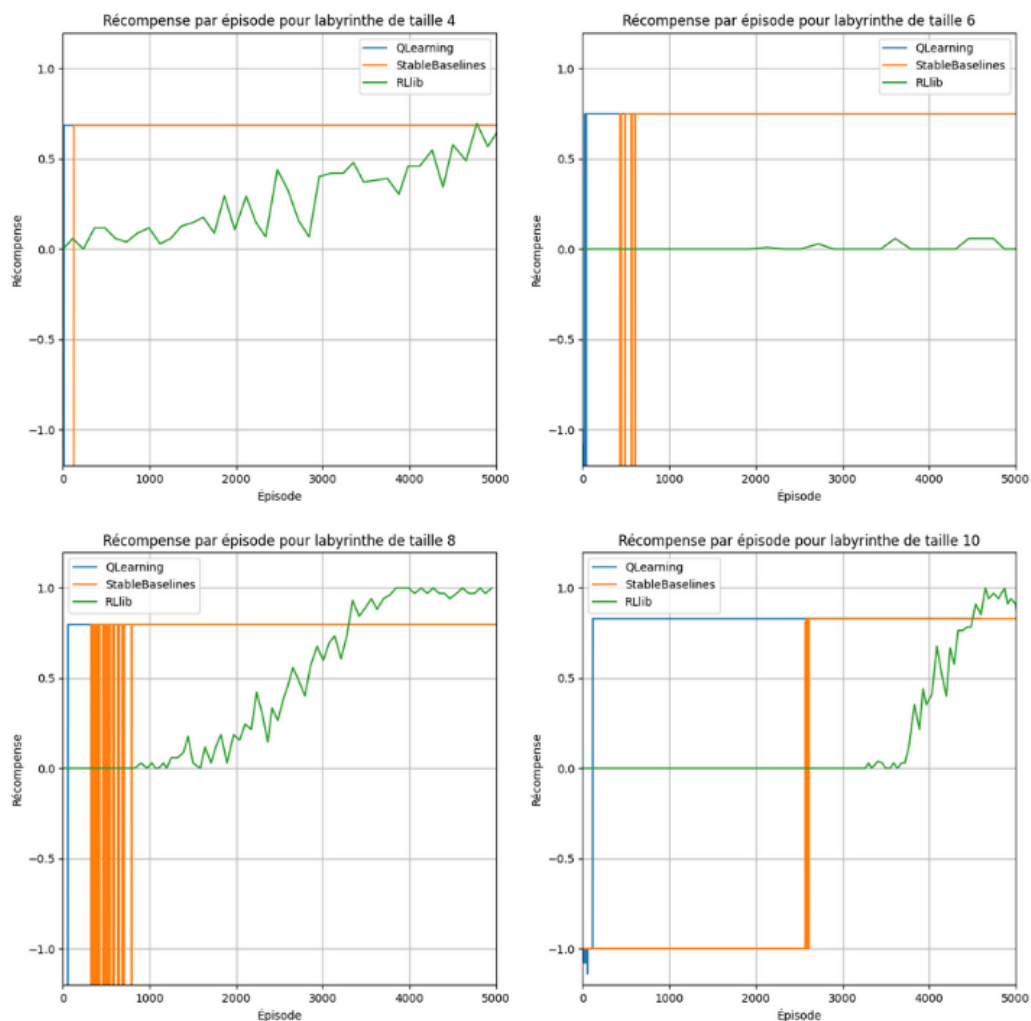


FIGURE 4.11 – Labyrinthe dernière comparaison

Lorsque l'on utilise un unique noeud dans la couche d'entrée du réseau pour représenter une information, il faut s'assurer que l'information codée de cette façon ait un ordre de grandeur.

Pour s'en assurer on peut utiliser la phrase : "J'ai deux fois plus de ... dans cet état que dans un autre."

Ici un noeud code une coordonnée de l'agent et il est simple de remarquer que la phrase "J'ai deux fois plus de coordonné x dans cet état que dans un autre." n'a aucun sens.

Cette façon de coder l'information est donc inadaptée, d'où les mauvaises performances.

4.2.5 Dernière comparaison

Voici une dernière comparaison : 4.11

Nous y comparons les performances du meilleur cas (meilleur paramétrage dans la meilleure version du labyrinthe) pour le QLearning et les deux implémentations de DQN.

Nous remarquons que le QLearning est bien plus efficace et que l'implémentation de RLlib de DQN ne permet d'atteindre de bonnes performances.

Il est important de noter que la mesure des performances du DQN de RLlib indique que ses performances dépassent l'optimal, il y a donc un souci qui se cache.

4.3 Livraison

Le problème de la livraison repose en grande partie sur le même principe que celui du labyrinthe.

De multiples versions ont été essayées :

- Dans la première, la possession de la boîte et la position représentent une unique information.
- Dans la seconde, la possession de la boîte et la position de l'agent sont deux entiers distincts (il y a donc deux OneHotEncoding pour le réseau, un pour la position, un pour la possession)
- Dans la dernière, la position est codée à l'aide de deux noeuds pour les coordonnées.

Il n'est pas étonnant que les résultats observés soient similaires à ceux du labyrinthe : 4.12.

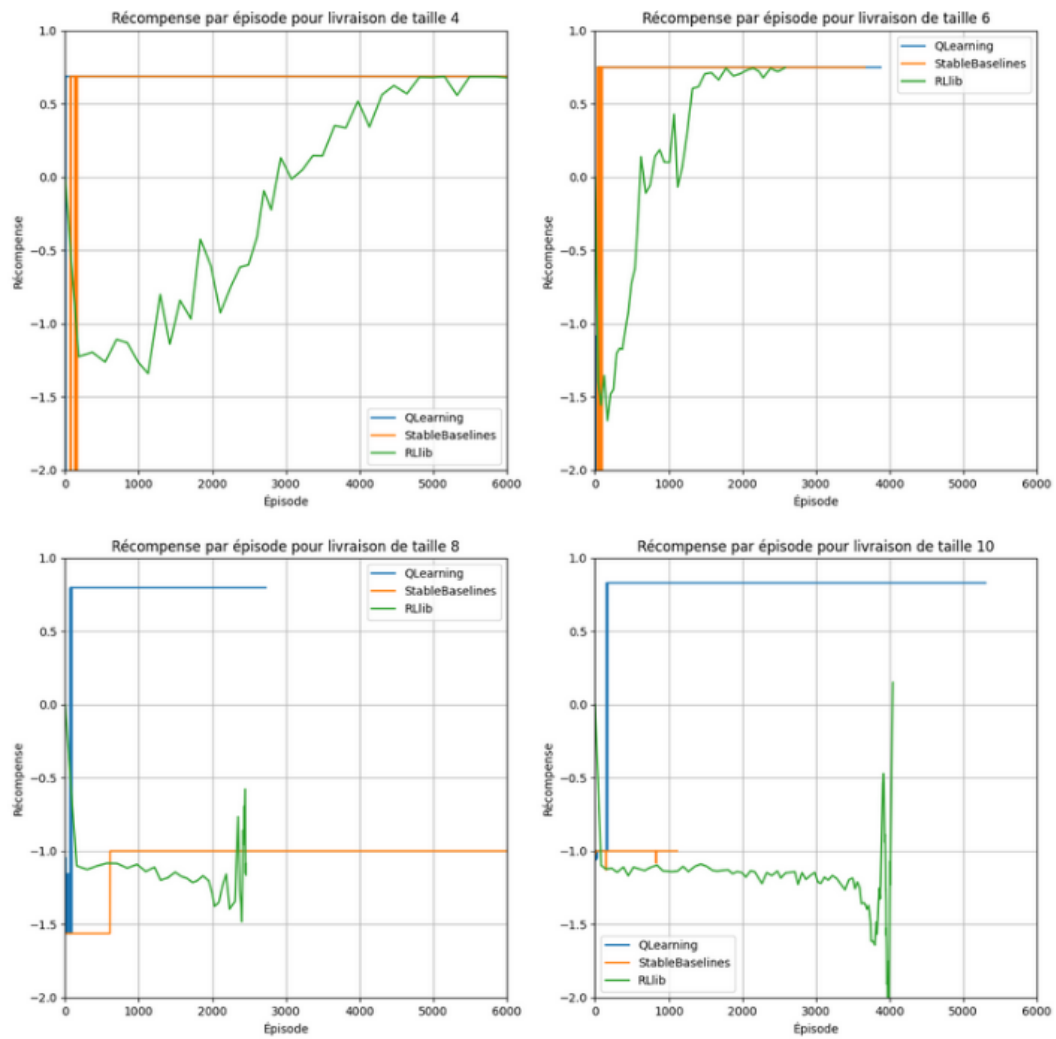


FIGURE 4.12 – Livraison comparaison des performances

Chapitre 5

Jeux avec adversaires

Le dernier problème traité est celui de RomeAlésia.

Le premier objectif est de battre un adversaire ayant un comportement fixé. Le QLearning suffit amplement pour cela, nous pouvons atteindre plus de 90 pourcents de victoire face à un comportement déterminé en conservant les paramètres par défaut de notre implémentation du QLearning.

Un problème plus complexe serait de créer un agent capable de battre un être humain.

Comme l'être humain possède un comportement qui évolue au fil du temps, on sort du cadre théorique du RL.

On peut donc considérer que l'opposant à notre agent peut jouer de multiples fois contre notre agent pour mieux comprendre comment ce dernier se comporte.

QLearning ainsi que DQN n'apprennent que pendant l'entraînement, à l'issue de celui-ci leur comportement est fixe.

De plus les comportements obtenus par QLearning et DQN sont prévisibles : dans un état fixé, ils feront toujours la même action.

Cela est problématique puisqu'un jeu comme RoleAlésia peut devenir un jeu de "bluff" assez rapidement.

On sait donc que :

- À la fin de l'entraînement, l'agent n'apprend plus.
- À la fin de l'entraînement, dans un état fixé, l'agent effectue toujours la même action.

Comme l'opposant apprend au fil du temps, alors s'il joue contre l'agent après l'entraînement, l'opposant pourra enregistrer ce que joue l'agent dans chaque état. Puis l'opposant pourra jouer de la même façon que l'agent, de cette façon le front restera au centre jusqu'à épuisement des unités.

Face à un agent QLearning ou DQN après son entraînement, il est possible de trouver un comportement qui force le match nul.

En l'état, ni le QLearning, ni DQN (quelque soit la façon dont paramètre/entraîne) ne permettra de battre un joueur humain à RomeAlésia sur de nombreuses parties.

Pour résoudre ce problème, il faudrait soit :

- Entraîner l'agent contre le joueur.
- Avoir un agent qui attribue des probabilités à chaque action et choisie aléatoirement en fonction de cette distribution.

Appliquer au moins un de ces points permet de ne plus avoir les hypothèses à propos du comportement de l'agent qui permettent de trouver une stratégie (jouer la même chose) qui force l'égalité.

Chapitre 6

Conclusion

6.1 QLearning et DQN

Ces deux algorithmes reposent sur le même principe mais n'utilisent pas la même structure de données pour représenter leur estimation de la fonction qualité.

Le QLearning possède moins de paramètres et est plus simple à paramétrer car plus stable.

Sur des problèmes où les espaces d'état et d'action sont trop grands, il est possible que le QLearning soit trop coûteux en mémoire.

DQN est plus instable, ce qui le rend plus complexe à paramétrer.

Néanmoins il peut généraliser ce qu'il observe.

Cela est très pratique dans certains problèmes (comme le couloir) mais pas du tout efficace dans d'autres (le labyrinthe).

Attention, les problèmes les plus "complexes" ne sont pas nécessairement ceux qui nécessitent DQN.

DQN peut prendre en entrée des observations continues ce qui est impossible avec le QLearning.

Ces deux algorithmes ont tout de même des limites, ils ont besoin d'un grand nombre de steps d'entraînement pour apprendre et le comportement vers lequel

l'agent converge est toujours un comportement prévisible : l'agent effectue dans un état fixé toujours la même action.

6.2 StableBaselines et RLlib

Nous avons vu que les performances de StableBaselines étaient supérieures à celles de RLlib.

StableBaselines est bien plus simple à utiliser que RLlib qui entremêle une ancienne et une nouvelle API. L'implémentation de DQN de StableBaselines (plus particulièrement de l'expérience replay) semble meilleure par rapport à celle de RLlib.

StableBaselines est par défaut, plus rapide que RLlib.

StableBaselines est mieux maintenu que RLlib où des sujets importants restent sans réponses sur le git pendant 5 ans.

Il faut préciser que StableBaselines a été utilisé sans calcul parallèle. Cela est beaucoup trop long pour RLlib au point où utiliser le calcul parallèle est obligatoire.

Mais il semblerait que le fait d'utiliser le calcul parallèle pour obtenir une vitesse d'entraînement acceptable pose de nouveaux problèmes ce qui explique peut-être les mauvaises performances.

Néanmoins RLlib possède un atout majeur, il permet de faire l'apprentissage par renforcement sur des systèmes multi-agents (pouvoir entraîner plusieurs agents simultanément dans le même environnement) ; cela n'est pas faisable avec StableBaselines.

De plus, RLlib donne plus de possibilités pour intégrer notre propre code. On peut créer notre propre implémentation de l'expérience replay par exemple.

Références

- [Richard Sutton, 2018] RICHARD SUTTON, A. B. (2018). Reinforcement learning : An introduction, 2nd edition. *The MIT Press*.
- [Stuart Russel, 2009] STUART RUSSEL, P. N. (2009). Artificial intelligence : A modern approach, 3rd edition. *Pearson*.
- [Volodymyr Mnih, 2013] VOLODYMYR MNIH, K. K. (2013). Play atari with deep reinforcement learning. *NIPS*. [Cité en page 14]
- [Watkins, 1989] WATKINS, C. (1989). Learning from delayed rewards. *PhD Thesis, University of Cambridge*. [Cité en page 8]